Задание на курсовое проектирование
по дисциплине Технологии разработки программного обеспечения

Цель работы:

Изучить теоретические принципы унифицированного процесса разработки СОИУ и составляющих его этапов. Получить практические навыки применения шаблонов при проектировании и разработке СОИУ. Освоить применение CASE средств для разработки СОИУ.

Исходные данные для выполнения КР:

- Описание исходных требований к создаваемой СОИУ (выдается преподавателем),
- Перечень паттернов для реализации (совпадает с вариантом ДЗ),
- Паттерн Gof (выдается преподавателем).

Порядок и время проведения работы:

Работа выполняется самостоятельно в часы внеаудиторных занятий. Выполняется студентом индивидуально. По итогам выполнения КР составляется РПЗ и проводится демонстрация работающей программы. Для защиты работы составляется презентация.

Задание:

Выполнить разработку СОИУ в соответствии с описанием ее функциональности (определяется вариантом) на основе моделей унифицированного процесса (RUP). Написать программу, реализующую фрагмент СОИУ, и реализовать в ней паттерны бизнес-логики, работы с БД и gof (по варианту). Для построения диаграмм использовать среду STARUml или ее аналог, например, Software Ideals Modeler. Ниже перечислены этапы разработки и их задачи, выполняемые в КР:

1 **Этап анализа и планирования требований (Начало) – срок 4 неделя:**

- Определить область применения СОИУ (предназначение, границы, интерфейсы с внешней средой, критерии сдачи-приемки).
- Определить прецеденты, критические для системы (основные функции и главные решения).
- Определить основные элементы архитектуры (для выполнения основного сценария).
- Определить и оценить самые опасные риски (угрожающие успеху разработки), предложить способы управления ими.
- Оценить затраты, длительность и стоимость разработки по модели COCOMO-2 этапа композиции приложения.

В пояснительную записку (рабочие материалы на 4 неделе):

1. Постановка задачи (задание по варианту).
2. Спецификация основных проектных требований, ключевых характеристик и главных ограничений (перечень функциональных и нефункциональных требований).
3. Модель предметной области (диаграмма классов предметной области и глоссарий понятий).
4. Выявленные актеры.
5. Выявленные прецеденты, их приоритеты и описание (кратко).
6. Диаграмма основных прецедентов.
7. (расширенная) Описание возможной архитектуры (перечень архитектурно-значимых прецедентов, обобщенные механизмы проектирования, системное ПО, используемые компоненты и/или каркасы). Диаграммы уровней подсистем и развертывания.
8. (дополнительно) По каждой из выбранных технологий (обобщенные механизмы

проектирования, системное ПО и т.д.) необходимо привести варианты, их достоинства и недостатки и обосновать выбор.

9. Перечень критических рисков и рекомендации по управлению.
10. Перечень экранных форм и их сложность; экспертные оценки скорости разработки и масштабных факторов; затраты, длительность и стоимость разработки:
    - дополнительная и расширенная - по модели COCOMO-2 этапа композиции приложения;
    - базовая - из ДЗ-1 ТРПО.

2 Этап проектирования (Развитие) – срок 8 неделя.

Для архитектурно-значимых прецедентов выполнить:

1. Составить модель требований (выделить актеров и прецеденты; создать прототип пользовательского интерфейса; детализировать и структурировать прецеденты).
2. Составить модель анализа (анализ архитектуры - выделить пакеты анализа и сервисные пакеты, определить классы сущностей и общие специальные требования; анализ прецедентов — определить классы анализа и их взаимодействие; анализ классов — определить ответственности, атрибуты и связи классов; анализ пакетов – определить состав и зависимости пакетов).
3. Составить модель проектирования (определить узлы и сетевые конфигурации, подсистемы и интерфейсы между ними, архитектурно-значимые и активные классы, обобщенные механизмы проектирования).
4. Выбрать и обосновать структурные шаблоны проектирования (MVC, PCMEF и т. д.)
5. Реализовать базовый уровень архитектуры на основе модели проектирования с применением паттернов базы данных и бизнес-логики.
6. Отслеживать риски, устранить наиболее серьезные.
7. Составить план итераций следующего этапа.

Программная реализация (макет на 8 неделе):

8. Работающая программа, реализующая основные прецеденты (базовый уровень архитектуры) и паттерны базы данных и бизнес-логики:
    - (базовая) - реализация CRUD или 1 основного прецедента и 1 паттерна;
    - (расширенная) - реализация (макет) базовых прецедентов и 2-х паттернов.

В пояснительную записку (рабочие материалы на 4 неделе):

1. Уточненная диаграмма (ы) прецедентов.
2. Описания (спецификации) прецедентов (предусловие, запуск и инициатор, порядок выполнения и поток событий, постусловия, запрещенные и альтернативные пути, атрибуты и ресурсы):
    - (дополнительная) - по трем прецедентам (один обязательно основной),
    - (расширенная) - по одному прецеденту (основному),
    - (базовая) - только краткое описание трех прецедентов.
3. Прототип пользовательского интерфейса (баз. - 2 шт., расш. - 3-4 шт., доп. от 5).
4. Диаграммы классов анализа: граничных, управляющих, сущностей.
5. Диаграммы взаимодействия для основных прецедентов (диаграммы классов-участников и диаграммы последовательностей):
    - базовая - диагр. классов (по 3-м основным прецедентам) и последовательностей (по одному основному, нетривиальная);

- расширенная, дополнительная - диагр. классов и последовательностей (по 3-м основным прецедентам, нетривиальные);

6. Пакеты анализа и сервисные пакеты (их состав и зависимости) в форме обобщенной диаграммы классов.
7. Распределение классов проектирования по подсистемам.
8. Трассировка пакетов в подсистемы, классов анализа в классы проектирования.
9. Диаграмма уровней подсистем.
10. Диаграмма размещения (развертывания) подсистем.
11. (дополнительно) - Диаграмма распределения классов по подсистемам с указанием интрефейсов подсистем.
12. (дополнительно) - Диаграммы взаимодействия для главного прецедента: одна диаграмма взаимодействия в терминах подсистем и их интерфейсов (укрупненное представление); диаграммы взаимодействия для каждой подсистемы в терминах ее классов (детализируют выполнение функции интерфейса).
13. Трассировка подсистем в компоненты, классов проектирования в исходные файлы.
14. Зависимость компонентов от исходных файлов.
15. (дополнительно) - Диаграмма размещения компонентов (с интерфейсами).
16. (расширенная) - Переработанный список рисков.
17. Перечень и состав итераций следующего этапа с указанием прецедентов, кооперaций, пакетов, подсистем и компонентов для разработки.

3 Этап построения (Конструирование) – срок 12 неделя:

1. Полностью определить и реализовать прецеденты.
2. Завершить анализ, проектирование, реализацию и тестирование.
3. Применить паттерны (по варианту) бизнес-логики, работы с БД и Gof.
4. Провести тестирование и отладку.

Этап построения содержит несколько итераций. Базовая и расширенная - одна итерация. Дополнительная - две итерации (вторая реализует GOF). Каждая итерация реализует определенный ранее набор прецедентов (сценариев). Для расширенной части - реализация (эмуляция) основных прецедентов.

Программная реализация (итоговое ПО на 12 неделе):

9. Работающая программа.
10. Программа должна содержать реализацию паттернов проектирования (по варианту).

В пояснительную записку по этапу (рабочие материалы на 12 неделе):

По итерации 1:

1. Экранные формы работающей программы.
2. Исходный код программы, реализующей архитектурно-значимые прецеденты и паттерны.
3. Полная диаграмма классов реализации.
4. Диаграммы последовательностей для иллюстрации работы паттернов.
5. Перечень и последовательность проведения тестов (модульные, интеграционные и системные) и тестовые варианты:
   - (базовая) — примеры функциональных тестов (по 1 основному прецеденту);
   - (расширенная) - примеры модульных тестов (по 1 основному прецеденту);
   - (дополнительно) - примеры системных тестов.

По итерации 2 (дополнительно):

6. Перечень новых прецедентов для реализации.
7. Уточненные диаграммы (по новым прецедентам):
   - прецедентов;
   - классов анализа и пакетов;
   - классов проектирования и подсистем;
   - компонентов и исходных файлов;
   - размещения подсистем и компонентов;
   - трассировки.

4 Этап внедрения (Переход) – срок 14 неделя:

- Применить ПО в среде заказчика.
- Завершить реализацию продукта.

В пояснительную записку (рабочие материалы на 14 неделе в составе РПЗ):

1. Перечень программ и рекомендации по установке.
2. Перечень документации для пользователей и заказчиков.
3. Рекомендации по внедрению (консультации, обучение и т.д.).

Сроки и отчетность:

Этап	Вид отчетности	Срок защиты	Баллы
Задание	Подписанное задание (на бланке) в 2-х экземплярах	22 февраля	
Начало	Отчет и рабочие материалы по этапу	4 неделя	15
Развитие	Отчет и рабочие материалы по этапу Работающая программа с паттернами бизнес-логики и БД	8 неделя	25
Конструирование	Отчет и рабочие материалы по этапу Работающая программа с паттернами бизнес-логики, БД и GOF	12 неделя	30
Переход	Расчетно-пояснительная записка Графические листы	14 неделя	10

Бонусная программа:

- Требования на уровень ниже.
- РПЗ и модели до конца марта.
- Защита в конце марта - начале апреля.

Защита курсовой работы:

К защите студент представляет (оценка до 80 баллов):

1. Расчетно-пояснительную записку на основе рабочих материалов.
2. Работающую программу и исходный код.
3. Графические листы (или презентацию) - базовая часть:
   - Листы с результатами этапа Начало (Требования к системе; Диаграмма прецедентов; Расчет затрат, стоимости, длительности; Риски и рекомендации по управлению ими).
   - Лист – диаграмма пакетов и классов анализа.
   - Лист – диаграмма классов проектирования (с подсистемами).
   - Лист – диаграмма уровней подсистем.
   - Лист – диаграмма развертывания (с подсистемами).

- Лист – перечень и примеры тестов, шаги по внедрению.

4. Графические листы (или презентация) - расширенная часть:
   - Листы с результатами этапа Начало (Требования к системе; Диаграмма прецедентов; Расчет затрат, стоимости, длительности; Риски и рекомендации по управлению ими).
   - Лист – диаграмма пакетов и классов анализа.
   - Лист – Трассировка пакетов в подсистемы.
   - Лист – диаграмма классов проектирования (с подсистемами).
   - Лист – диаграмма уровней подсистем.
   - Лист – диаграмма последовательностей для иллюстрации работы паттернов.
   - Лист – диаграмма развертывания с указанием подсистем.
   - Лист – перечень и примеры тестов (модульные и функциональные), шаги по внедрению.
5. Графические листы (или презентация) - дополнительная часть:
   - Листы с результатами этапа Начало (Требования к системе; Диаграмма прецедентов; Расчет затрат, стоимости, длительности; Риски и рекомендации по управлению ими).
   - Лист – уточненная диаграмма прецедентов.
   - Лист – диаграмма пакетов и классов анализа.
   - Лист – диаграмма классов проектирования (с подсистемами).
   - Листы – диаграммы последовательностей для иллюстрации работы всех паттернов.
   - Лист – диаграмма уровней подсистем.
   - Лист – диаграмма развертывания с указанием компонентов и их интерфейсов.
   - Лист – трассировка пакетов в подсистемы, подсистем в компоненты.
   - Лист – перечень и примеры тестов (модульные, функциональные и системные).
   - Лист – шаги по внедрению.

В процессе защиты студент (оценка до 20 баллов):

1. Делает доклад по листам/презентации (до 5 минут).
2. Демонстрирует работающую программу.
3. Отвечает на вопросы по программе и РПЗ и на общие теоретические вопросы.

Оценка курсовой работы:

- Базовый вариант - до 70 баллов (удовлетворительно);
- Расширенный вариант - от 71 до 84 баллов (хорошо);
- Дополнительный вариант - от 85 до 100 баллов (отлично).

Литература основная:

1. Унифицированный процесс разработки программного обеспечения: учебное пособие / Виноградова М.В., Белоусова В.И. – М.: МГТУ им.Н.Э. Баумана. – 2015 г. http://ebooks.bmstu.ru/catalog/193/book1303.html
2. Гамма Э. и др. Приемы объектно-ориентированного проектирования. Паттерны проектирования. - Спб.:Питер. - 2009 г.
3. Фаулер М. Архитектура корпоративных приложений. - М.:Изд.дом Вильямс. - 2008 г.

Литература дополнительная:

4. Орлов С.А. Технологии разработки программного обеспечения. - СПб: Питер, - 2002 г.
5. Якобсон А, Дуч Г., Рамбо Дж. Унифицированный процесс разработки программного обеспечения. - Спб.: Питер. - 2002 г.